

Winter Milestone Compliance Report 2

GK02: Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Project Manager:

Cameron Tuffner-Lyons - 501029609

Team Members:

Victor Do - 501137174

Davis Cheung - 501014636

JJ Marr - 500879103

Faculty Supervisor

Dr. Gul N. Khan

February 13th, 2025

Abstract:

This interim report outlines the progress made in developing a real-time heterogeneous CPU-GPU fluid simulation within Unity. The project aims to improve computational efficiency by optimizing workload distribution between the CPU and GPU using Unity's compute shaders and job system. We have explored key architectural considerations, including CPU-GPU communication protocols, PCIe-based data transfer optimization, and memory management techniques to reduce performance bottlenecks. Several enhancements have been implemented, such as temperature diffusion, state change triggers, dynamic particle management, and new simulation boundary behaviors, all of which contribute to improved realism and efficiency. Additionally, we have introduced gameplay and structural improvements, including a tooltip system, prefab-based extensibility, and a level management framework to streamline content creation. While significant progress has been made, ongoing challenges such as memory fragmentation, data synchronization inefficiencies, and computational overhead from interparticle interactions remain areas for further refinement. The next phase of development will focus on optimizing hashing algorithms, expanding simulation features, and improving performance across different hardware platforms.

Tasks Outlined & Group Member Contributions:

Individual Tasks: Student A: (VD)	Individual Tasks: Student B: (JM)	Individual Tasks: Student C: (CTL)	Individual Tasks: Student D: (DC)
Functional Tasks Improved upon Level 2 design. Code refactors and animation updates. Added locomotion system, level management system, prefab system, tooltip system, multi-fluid implementation and other UI enhancements. Worked on Visual Shaders and obstacle interactions. Report Sections - Proofreading - Dynamic Particle Management Tools - Enhancements - Game Content	Functional Tasks Added circle collider support Unity 6 and WebGPU support	Functional Tasks Level 2 design and implementation. Large collider - small collider demonstration level. (WIP) CPU-GPU data transfer minimization. Theory and Design Report. Optimized Particle readBacks Report Sections Architecture PCIe Simulation Boundaries Abstract/Conclusion	Functional Tasks Thermal diffusion kernel State change kernel Thermal objects and thermal fluid properties Counting sort refactor Particle array writeback refactor Report Sections Unity dispatching Temperature diffusion State change Triggers

Table 1: This is a breakdown of the specific tasks each member worked on.

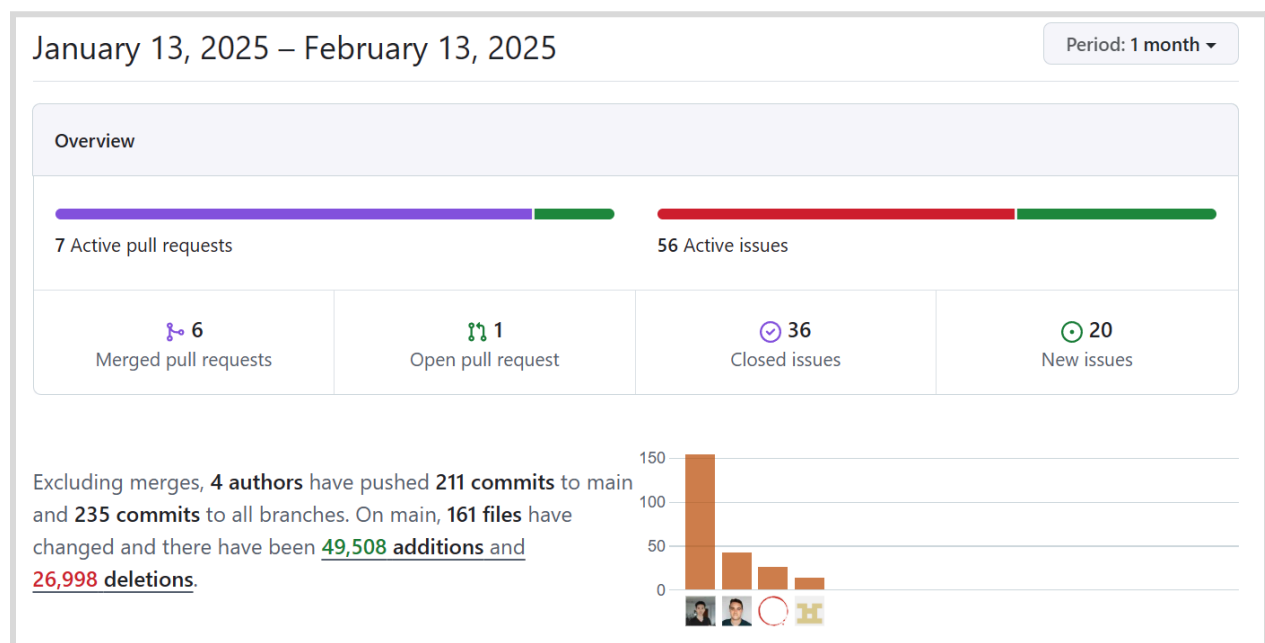


Figure 1: This is a breakdown of closed issues and pull requests over the past month.

Architecture:

CPU-GPU Communication Protocols

One of our goals for this MCR was to develop a stronger understanding of the communication protocols that govern CPU GPU interactions and data transfers and leverage this information to analyze and improve on our heterogeneous CPU-GPU algorithm. Due to the varied nature of system integrations, this section will cover the main communication protocol/architecture used in modern computer systems.

Peripheral Component Interconnect Express - PCIe

The information in this section comes from [1]

The primary communication protocol used for CPU-GPU communication between a discrete GPU and CPU is the PCIe protocol. The PCIe protocol is a high-speed serial communication expansion bus used as an industry standard for connecting high-performance consumer CPUs with high-performance consumer discrete GPUs. PCIe was created as a replacement for older bus architectures such as PCI and AGP.

PCIe uses a point-to-point topology between the CPU and peripheral devices. Each device connected via PCIe is assigned a certain number of PCIe “lanes”, which are dedicated data channels implemented on the CPU silicon architecture. Each PCIe lane consists of two unidirectional channels, forming a full-duplex link, and lanes can be assigned in 1x, 2x, 4x, 8x, 16x, and 32x configurations depending on the number of lanes available and PCIe generation. This direct connection from the cpu to the peripheral device allows for high-speed parallel (if the device has more than one lane assigned to it) data transfers and reduced bus wait time.

Data transmission in the PCIe protocol uses a packet-based system, similar to computer networks. A packet is first received by the physical layer and converted to bits, which are then checked for errors using both sequence numbering and CRC, with retransmissions occurring for corrupted packets. The packets are then moved to the transaction layer which determines the type of transaction being processed (memory, I/O, or configuration space access) and handles the request.

The communication speed of a PCIe connection is determined by the PCIe standard implemented on the system; for the PCIe 5.0 standard, the communication speed is 4GB/s per lane. The typical number of lanes dedicated to a discrete GPU is 16 lanes, giving a total bandwidth of 64GB/s. From this communication speed it is possible to estimate a minimum data transfer cost for transferring the information of a single particle between the CPU and GPU on a system implementing the PCIe 5.0 standard. For example, if the information about a single particle equates to 100 bytes, then the minimum data transfer time (excluding any overhead processing) would be $\frac{100}{64 \times 2^{30}} \approx 1.455ns$.

Other on-chip CPU-GPU architectures and protocols.

Systems that use integrated CPU-GPU SoCs do not require a peripheral communication protocol to communicate, since both processors exist on the same silicon. Instead, The two processors communicate using the on chip high-speed communication bus(es) such as “Infinity Fabric” for AMD processors, Intel’s “Ring Bus”, or Apple’s “Unified Memory Architecture”. The tightly integrated nature of these integrated CPU-GPU SoCs means that the communication between the two processors is not necessarily an open standard. However, these types of SoCs typically implement a shared memory architecture where the system memory is shared between the CPU and GPU. This removes the need for large data transfers between the two processors since both have access to the same memory and instead communication is handled through high-speed on-chip interconnects. Apple’s unified memory architecture is an example of one of these implementations.

Unity Dispatching

During the development process for our heterogeneous algorithm, we realized that we needed to know how dispatching worked on the Unity platform for both CPU and GPU kernels. We wanted to better understand how and when data transactions took place, how the workload was actually dispatched onto each device, as well as what underlying API’s were being used. The following section describes our findings and describes Unity dispatching libraries for Compute Shaders (for GPU) as well as the Jobs System (for CPU).

Compute Shaders (GPU Kernels)

At a high level, the Compute Shader system is an HLSL-based dispatching system for executing tasks in parallel across multiple threads. Like other shader languages, it is comprised of a series of kernels, which can be called from the game’s main C# script. Work is divided into user-defined thread groups on each dispatch call to a given kernel (defined in HLSL when creating them Kernel), and users must define memory buffers ahead of time as data structures within both C# and HLSL code (C#-side definition acts as a translation layer for the Compute Shader) [1].

At a low level, Unity’s dispatch calls translate directly into several different equivalent dispatch calls depending on the targeted platform. By default, this is generally reducible into Direct3D for DirectX (Windows) platforms, Metal for iOS, and Vulkan for Android, however a full list may be seen below [1]:

- Windows, Xbox (Direct3D)
 - **Analogous API call(s):**
 - i. *ID3D11DeviceContext::Dispatch (DX11)*
 - ii. *ID3D12GraphicsCommandList::Dispatch (DX12)*
 - **Dispatch Process**

- i. Unity compiles the Compute Shader into HLSL and creates a shader program.
 - ii. Unity binds the shader program, resources (e.g., buffers, textures), and pipeline state.
 - iii. Unity issues a Dispatch command, specifying the number of thread groups in X, Y, and Z dimensions.
 - iv. The Direct3D runtime sends the command to the GPU driver, which schedules the compute shader execution on the GPU.
- iOS, MacOS (Metal)
 - **Analogous API call(s):**
 - i. *MTLComputeCommandEncoder::dispatchThreadgroups*
 - **Dispatch Process**
 - i. Unity compiles the Compute Shader into Metal Shading Language (MSL).
 - ii. Unity creates a compute pipeline state and binds it to the command encoder.
 - iii. Unity binds resources (buffers, textures) to the command encoder.
 - iv. Unity issues a dispatchThreadgroups command, specifying the number of thread groups and threads per thread group.
 - v. The Metal runtime submits the command buffer to the GPU.
- Vulkan (Cross-Platform; generally mobile/Android)
 - **Analogous API call(s):**
 - i. *vkCmdDispatch*
 - **Dispatch Process**
 - i. Unity compiles the Compute Shader into SPIR-V (Vulkan's intermediate representation).
 - ii. Unity creates a compute pipeline and binds it to the command buffer.
 - iii. Unity binds descriptor sets (resources like buffers and textures) to the command buffer.
 - iv. Unity issues a vkCmdDispatch command, specifying the number of workgroups in X, Y, and Z dimensions.
 - v. The Vulkan driver submits the command buffer to the GPU for execution.
- OpenGL (Legacy)
 - **Analogous API call(s):**
 - i. *glDispatchCompute*
 - **Dispatch Process**
 - i. Unity compiles the Compute Shader into GLSL.
 - ii. Unity binds the shader program and resources (e.g., buffers, textures).
 - iii. Unity issues a glDispatchCompute command, specifying the number of workgroups in X, Y, and Z dimensions.
 - iv. The OpenGL driver submits the command to the GPU

Jobs System (CPU Kernels)

CPU dispatching for Unity is more complicated. By default, the C# executing background game functions (such as calling the dispatcher for Compute Shaders, handling user input, etc.) is a single-threaded script that performs limited tasks and lacks support for formal CPU dispatching of parallel jobs. Additionally, base Unity lacks thread safety, therefore any implementation in dry C# is plainly unfeasible. For this reason, we use the Unity Jobs System.

At a high level, the Jobs System is a C#-based CPU multithreading system designed to enable Unity scripts to execute code in parallel across multiple cores. It provides an array of thread-safe data structures and functions that enable asynchronous, thread-safe, parallel execution. Jobs are created through the use of the *Job* struct, a managed structure implementing the *IJob* and *IJobParallelFor* interfaces, alongside *Native Containers*, unmanaged memory buffers which operate primarily in main memory. Dispatching jobs is as simple as calling a *Schedule* function with batch size, followed by a *Complete* function to ensure that the task is completed before further serial code execution takes place [3].

At a low level, the Jobs System is reliant on the *Burst Compiler* to optimize Unity script code into platform-native code that executes more efficiently than plain C#. The Burst Compiler is a part of Unity's Data-Oriented Technology Stack (DOTS), and utilizes the LLVM project to perform just-in-time compilation of highly-optimized machine code corresponding to user-written CPU kernels. By default, when a Jobs System game is run on Unity, the Burst Compiler asynchronously compiles all user-written kernels in native code optimized for the user's CPU alongside the game script at runtime. The list of specific optimizations is long and device-dependent, but the general flow is as such [3], [4], [5]:

1. **Input:** The Burst Compiler takes C# code as input.
2. **Intermediate Representation:** The C# code is converted into an intermediate representation that is easier to optimize.
3. **Optimization:** The intermediary code is optimized for performance via LLVM, including:
 - a. Loop unrolling
 - b. Vectorization (using SIMD instructions)
 - c. Dead code elimination
 - d. Constant folding
4. **Native Code Generation:** The optimized code is compiled into platform-specific native code (e.g., x86, ARM).
5. **Execution:** The native code is executed directly by the CPU

Additionally, Jobs System code generally behaves like Compute Shader code, where the programmer is limited to primitives, structs, and little to no libraries (only the Unity base Mathematics library, which is also built on the Jobs System) [FIXME].

Unity's dispatching mechanisms for both GPU and CPU workloads offer a flexible approach to heterogeneous computing within game development and real-time applications. Compute Shaders provide a structured way to offload parallel tasks to the GPU, while the Jobs

System enables efficient multi-threading on the CPU with native code optimizations. Proper understanding of the intricacies of both systems may allow us to optimize our algorithm's workload distribution and maximize performance across different hardware platforms, however careful memory management, scheduling, and knowledge of underlying platform APIs will be essential for achieving the best results when leveraging Unity's dispatching capabilities.

Game Features:

Temperature Diffusion:

One of the new features introduced in our current version of our game is a temperature simulation mechanism. To achieve this we introduced temperature diffusion; each particle's temperature is adjusted by the average of the temperature differences from all of its neighbours multiplied by a fluid-dependent constant specific to that particle, called its *thermal diffusivity*. Temperature diffusion can be visualized on particles with temperature-dependent shaders, notably lava (see Figure 2.1, 2.2). Note that particle contributions are proportional to the particle's smoothing radius, so closer particles have greater impact on the change in temperature. Static heating elements were also added through use of simple box colliders wrapped in a struct with temperature, range, and conductivity values.

Demonstration of temperature diffusion visualized through the lava's visual shader:

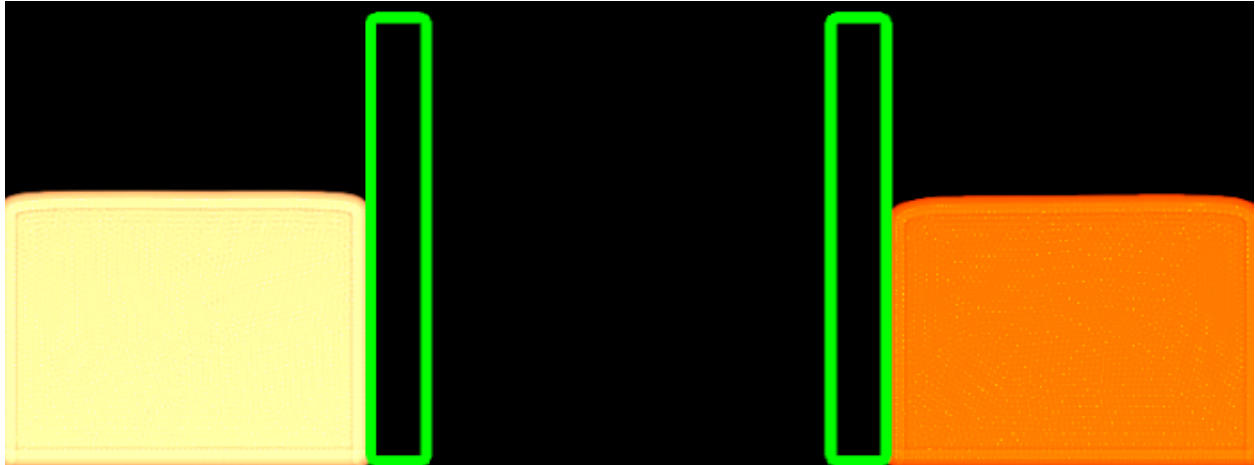


Figure 2.1: On the left is lava at 1200°C, on the right is lava at 700°C. They are separated by a physical barrier. You can see the temperature based black body radiation visualization displaying the two pools of lava as different colors.



Figure 2.2: When the physical barrier is removed the two pools of lava don't immediately homogenize or mix, they meet in the middle and their respective temperatures diffuse until they reach an equilibrium point which is the average of their two temperatures after a certain time based on their thermal diffusivity values.

State Change Triggers:

In addition to temperature diffusion, we also added a simple state change trigger. A couple different models were proposed, including an energy-based model that holds particles at the same temperature before performing a state transition, however such a model would require a much more computationally intensive kernel as well as a partial refactor of existing code. Instead, we settled on immediately transitioning particle states at temperature thresholds (e.g. water to steam at 100 degrees Celsius, Figure 3.1, 3.2), and instead tuned simulation settings until the state transitions for boiling or cooling fluids looked reasonable.

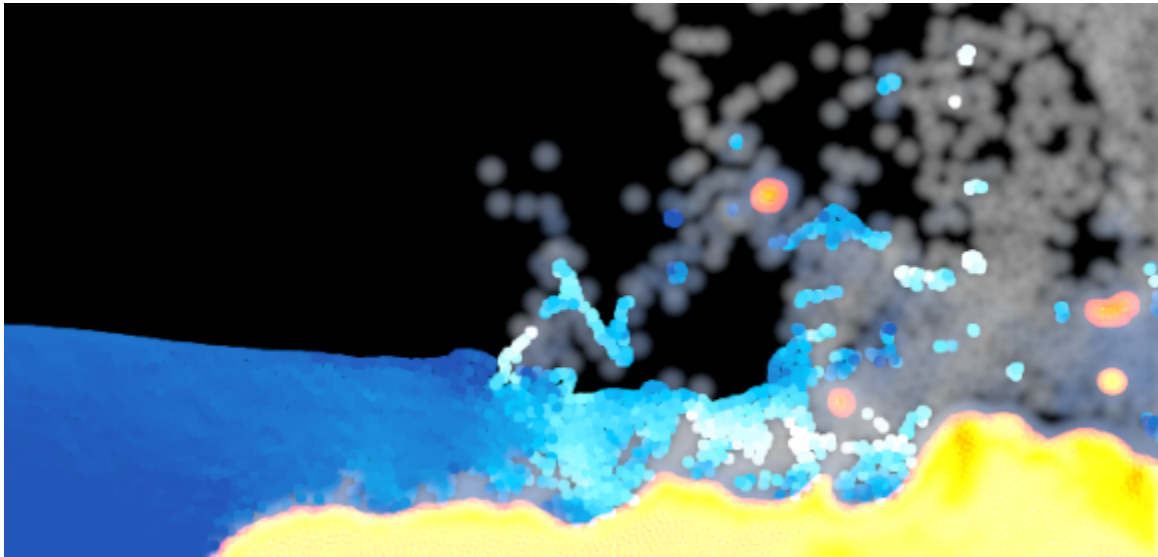


Figure 3.1: Water particles changing to steam when heated up via contact with lava particles

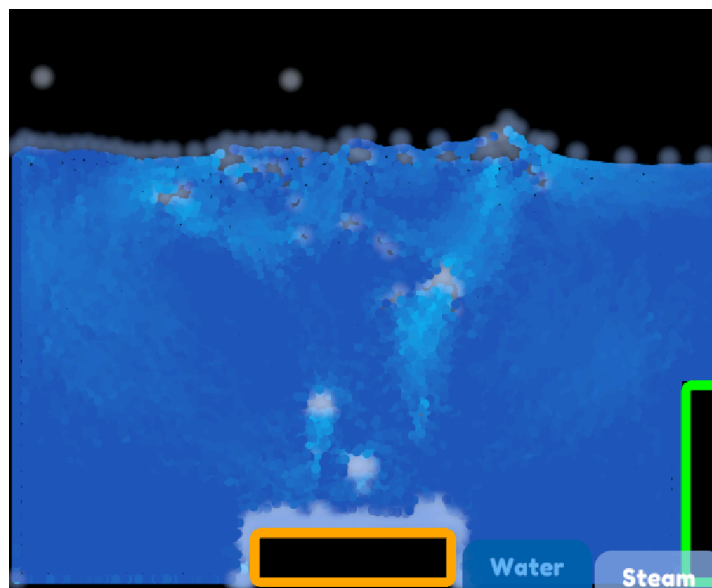


Figure 3.2: Water particles changing to steam when heated by a static heating element

Dynamic Particle Management Tools:

- Added Source Objects:

These are objects which produce particles. They take a transformation position and scale vector. You are able to set an initial velocity for the particle, the spawn rate for the source object and the fluid type that the particle should spawn as (Figure 4).

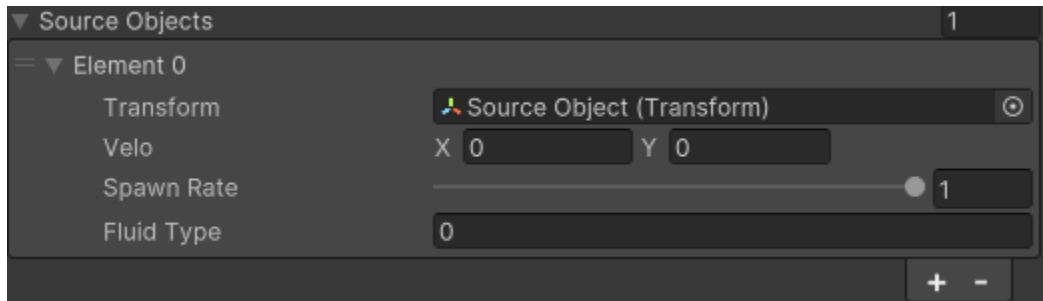


Figure 4: This image shows the variables that can be changed in the Unity inspector.

- Added Drain Objects:

These are objects that remove particles from the scene.

Particle Management Implementation Design Considerations:

In order to dynamically add and remove particles from our scenes we needed to take into consideration memory coherency as well as thread synchronization.

One might first attempt to allocate memory each time a particle is created or destroyed. But this approach is flawed as it would slowly increase memory fragmentation over time which leads to longer memory access patterns due to cache misses and this is the bane of optimization.

Instead, at the start of a scene a chunk of contiguous memory is allocated with the exact maximum number of particles structs that we expect to need for that scene. Each struct in this array corresponds to a particular particle to be rendered in the scene, the struct contains information about the particle's position, temperature, temperature, type and so on. The field corresponding to the particle's type is the most important for activating and deactivating particles. When that 4 byte int field is set to 0, then the particle will be ignored by most kernels including the visual shader which will hide the particle off screen. The particle type field also serves a dual purpose as it is used to set an ID which determines its visual and physical properties whenever it is not disabled. After creating this array of structs we then dispatch an equivalent number of threads as there are structs in the array. Each thread is responsible for one particle. Note that because threads on the GPU are grouped together in groups of 32 or 64 units called warps or wavefronts which execute the same instruction simultaneously on different data; it is a good idea to choose a multiple of 32 or 64 as the maximum number of particle for a scene, to minimize unused threads in warps/wavefronts.

Because each thread runs concurrently in parallel rather than consecutively that means each thread is unaware of the state of other threads or of the overall simulation. In our simulation, each particle of fluid is assigned its own thread which updates its position, temperature, state, and so on. Thus, if a command was sent to our compute shader to spawn some particles, each thread, acting independently, would reactivate the particle it is assigned to and the entire scene would immediately become flooded with particles. To avoid this we need to consider thread synchronization and race conditions.

Firstly we need to set an upper limit on the amount of particles which can spawn each frame to a reasonable amount. Then we must increment a counter for each particle that gets spawned during that frame. The problem is that if we use a regular variable for this each thread would read the value of that variable at the start of the frame which would be 0 and then set the value to 1, this is called a race condition, where multiple threads attempt to read and modify the same memory location simultaneously, leading to unpredictable and incorrect results. In this case, every thread would think it was the first to spawn a particle, causing us to exceed our per-frame particle limit.

To solve this, we need to use atomic operations. An atomic operation is one that cannot be interrupted by other threads while it's executing. By using an atomic increment operation on our spawn counter, we ensure that each thread gets a unique value. If that value is below our per-frame spawn limit, the thread can proceed to activate its particle. If the value exceeds the limit, the thread returns without spawning anything.

The atomic increment operation is implemented using hardware-level atomic instructions that guarantee thread safety. In our specific implementation with HLSL, we use `InterlockedAdd` which takes three parameters: a destination value to modify, a value to add, and an output variable that receives the original value before modification. This function ensures that when multiple threads attempt to increment our spawn counter simultaneously, each thread receives a unique value and the counter is incremented exactly once per thread without any race conditions. For example, if Thread A and Thread B both try to increment the counter when it's at 5, Thread A might get 5 as its return value and Thread B would get 6, while the counter ends up at 7 - exactly what we want.

A side effect of this approach is that we can get a consecutive number for each thread which tries to spawn a particle. Then after some modifications to the compute buffer dispatcher to send a number indicating the current frame of the simulation. We can put use these two numbers inside of a function similar to a hash called a PCG (permuted congruential generator) to generate a pair of pseudo random numbers which can then be used to randomize the position of the particle within a specific bounds, this will lower the density of the particles as they spawn which will increase performance because there will be less neighbour checks per particle thread, and it will allow for more better more stable spawn behavior as the particles would have a sudden spike in pressure from being spawned in a tight grouping.

Conversely, when a particle is disabled, whether it is through a drain object or any other means, its position must be randomized using that same PCG. This is because in a scenario where there is a fixed drain object in the scene and a source of particles, as the particles become disabled by the drain they will accumulate on its boundary. This is undesirable because as active particles approach this boundary they will have a sudden increased computational load as they have to perform additional neighbourhood checks and verifications for inactive particles. This can cause the frames per second to go from 400fps to 20fps suddenly due to lowered wave coherence.

Simulation Boundary Behaviors:

To increase variety in level design, three types of boundary behaviors were implemented: “Solid”, “Void”, and “Looping”.

Solid Boundary

The solid boundary type functions as a wall which the particles cannot pass through and instead will collide and bounce off. This boundary type allows for the creation of a closed system of particles, where no particles can leave the boundary. The implementation of this boundary exists as part of the collision handling GPU kernel. Each particle's position is compared to the boundary limits defined for the simulation, and if a particle is in violation of the simulation boundary it's position is modified to bring the particle back into the simulation boundary, and the velocity of the particle is “reflected” perpendicular to the boundary axis, along with some collision damping. This algorithm ensures that all particles remain within the simulation boundary and all particle-boundary collisions correctly update the particle's velocity, creating a more realistic simulation of a fluid inside a “container”.

Void Boundary

The void boundary type removes the simulation boundary and instead ensures that particles which have travelled past the boundary are no longer considered “active” and can thus be ignored during fluid calculations. This boundary algorithm is also implemented in the collision handling kernel on the GPU. Particle positions are compared with the simulation boundary and every particle that has violated this boundary is moved “disabled”. Disabling a particle prevents fluid calculations for that particle and ensures that the particle is not rendered in the scene.

Looping Boundary

The looping boundary type operates similarly to the void boundary type, but instead of disabling particles the looping boundary creates a spatial loop for each boundary. When a particle crosses the simulation boundary, its position is set to the location of the opposite boundary, effectively creating a loop. The particle's velocity is retained across the boundary, such that if you move a particle as a set velocity through one boundary, it will return with the same velocity from the opposite direction it was originally moving. The boundary algorithm was implemented in the collision handling kernel on the GPU and the result creates a looped finite space which the particles move through.

Enhancements:

- **Tooltip System:**
 - When hovering over elements in the game tooltips with useful information are displayed for increased clarity for the player (Figures 5.1-5.4).

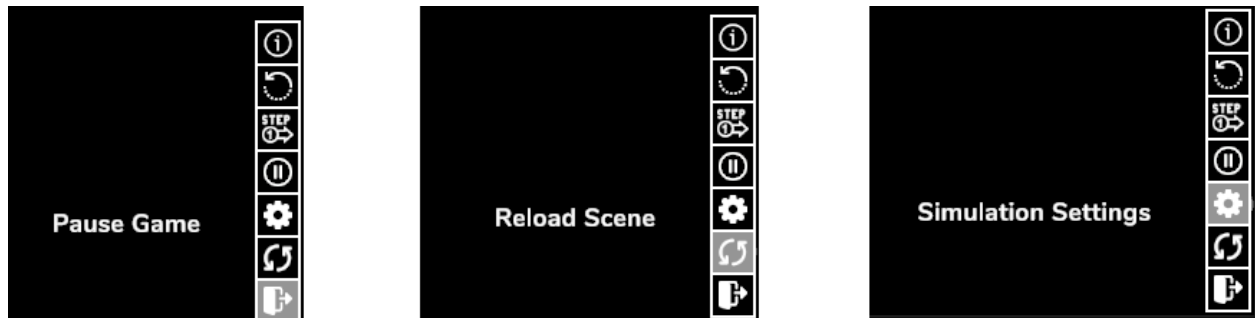


Figure 5.1, 5.2, 5.3: Showcase of the tooltip when hovering mouse over menu elements.

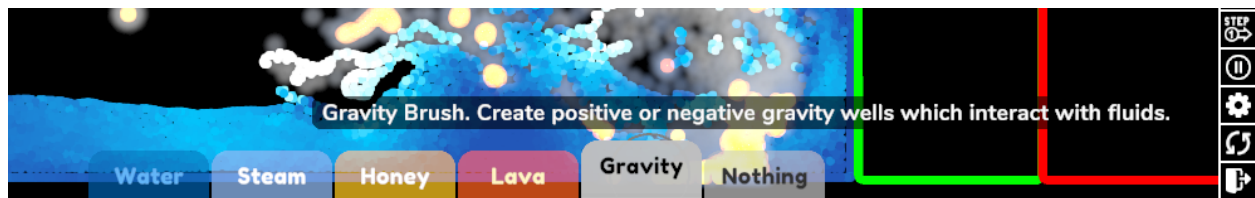


Figure 5.4: Tooltip showing description when hovering over fluid options.

- **Prefab System:**
 - The prefab system is an example of extensible design. Each level reads from a single set of definitions for common elements such as the UI. This makes the overall game more extensible because one edit to the common definition is immediately reflected in each level despite them being standalone implementations otherwise.
- **Level Management System:**
 - Each level needs a manager that ticks each frame and runs a series of game state checks to see if the level has been completed yet. We created an inheritable class which contains all the common elements like level complete animations and we extend that class for each level with the specific triggers that the manager for each level needs to listen to and keep track of.
- **Locomotion System:**
 - A common theme through many of the levels is automated movement of colliders and obstacles. Whether it is a door or a fan or propeller we created a series of scripts which can do back and forth movements, spinning movements, orbits, etc. And we made each of these scripts highly parameterized so we can easily create multiple instances of these scripts with different parameter values for their speed and timings.

Game Content:

Level design and implementation is an iterative process. Thankfully we have spent a lot of time developing extensible tools which we are now finally able to leverage in order to accelerate the development process. By taking advantage of these tools such as the prefab system, level management system, progression system, animation systems, and locomotion systems we were able to rapidly develop a second prototype level for our game.

Level 2: Carnival Chaos

- **Theme and Goal:** This level has a carnival theme, the goal of the level is to have the player aim a wobbly water gun to shoot a target and fill up a water meter. There will be rows of moving obstacles that occasionally block the line of sight for the player and the target will be moving as well. The player will be scored based on time to complete level and water wasted (Figure 6.1-6.3).
- **Status:**
Functional, Needs more refinements.
- **Input Method:**
Aim water gun at bottom of screen with mouse. Left click to shoot

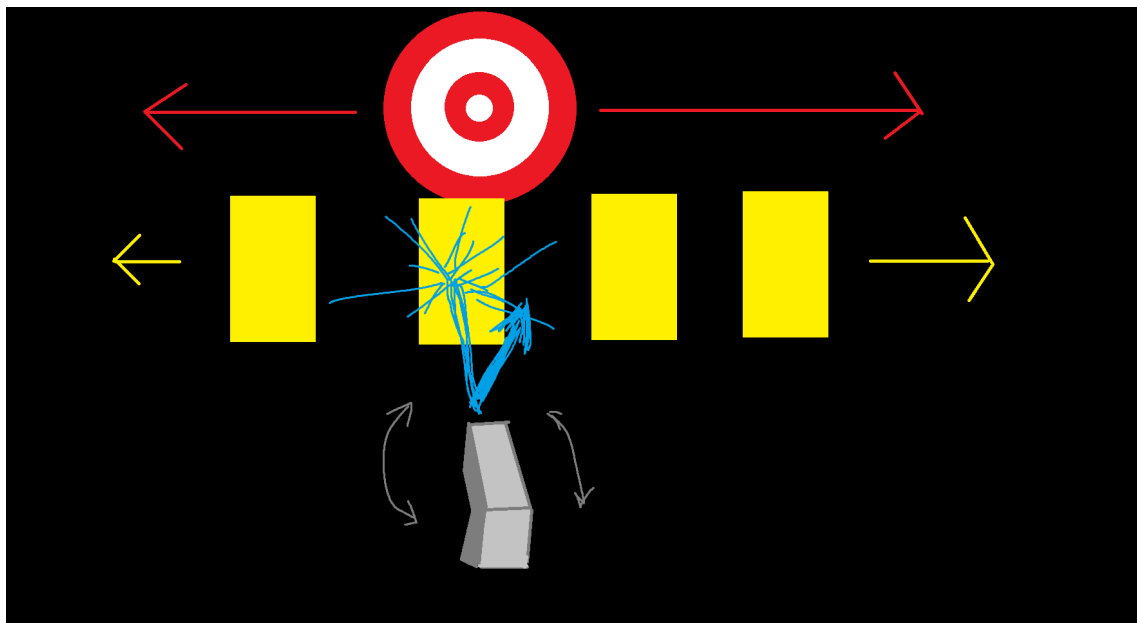


Figure 6.1: Preliminary sketch of the scene in MS Paint



Figure 6.2: Initial Partially Functional Mockup In Unity



Figure 6.3: Current Iteration of Scene, fully functioning in unity with placeholder art assets. The goal is to fill the bar on the left. The bar on the right is the remaining water in the gun before it needs to reload. The target and the obstacles slide left and right continuously.

Sandbox Scene:

This level which is accessed directly through the main menu was designed as a free play space where the player can use and experiment with all of the fluids and objects that we have developed for the simulation. It also serves as a benchmark level that we will use to evaluate the performance of the fluid simulation. This level will be updated to include more features as we develop the game, allowing us to test all features in a single environment, streamlining the testing process (Figure 7).



Figure 7: Showcase of Sandbox scene, with multiple fluids, and multiple box colliders.

Demonstration Scene:

When implementing collision detection between particles and colliders of varying sizes for our scenes, we observed that the small particles which constituted our fluid would far outnumber the number of larger colliders. We also noticed that the larger colliders would need to vary in shape and size to represent various objects whereas the smaller fluid particles would be more homogeneous and consistently shaped. This draws a parallel to the hardware architectures of the CPU and GPU. On one hand the CPU is built using a handful of large and powerful cores which can concentrate on different tasks in quick succession whereas the GPU is built using groupings of threads which concurrently execute similar tasks on different data in parallel. Thus, because of their architectures the GPU is particularly well suited for the task of simulating the fluid particles while the CPU is well suited for handling the surrounding obstacle colliders (Figure 8).

The key issue on the GPU stems from SIMD (Single Instruction, Multiple Data) execution in wavefronts. GPU threads within a wavefront must execute the same instructions simultaneously for optimal performance - this is called wave coherency. When dealing with colliders of drastically different sizes, the collision detection code paths can diverge significantly. Some threads might need to perform complex calculations for the large sphere, while others handle simpler checks for small colliders. This thread divergence breaks wave coherency and forces the GPU to serialize execution, significantly impacting performance.

Additionally, our spatial hashing grid presents another challenge. The grid cell size needs to be large enough to properly handle our biggest collider to ensure we don't miss any potential collisions. However, this larger grid size means that when checking for collisions with smaller colliders, particles must check many more neighboring cells than strictly necessary. Each particle ends up performing many unnecessary neighbor checks, as the spatial partitioning is optimized for the largest collider rather than the more numerous smaller ones.

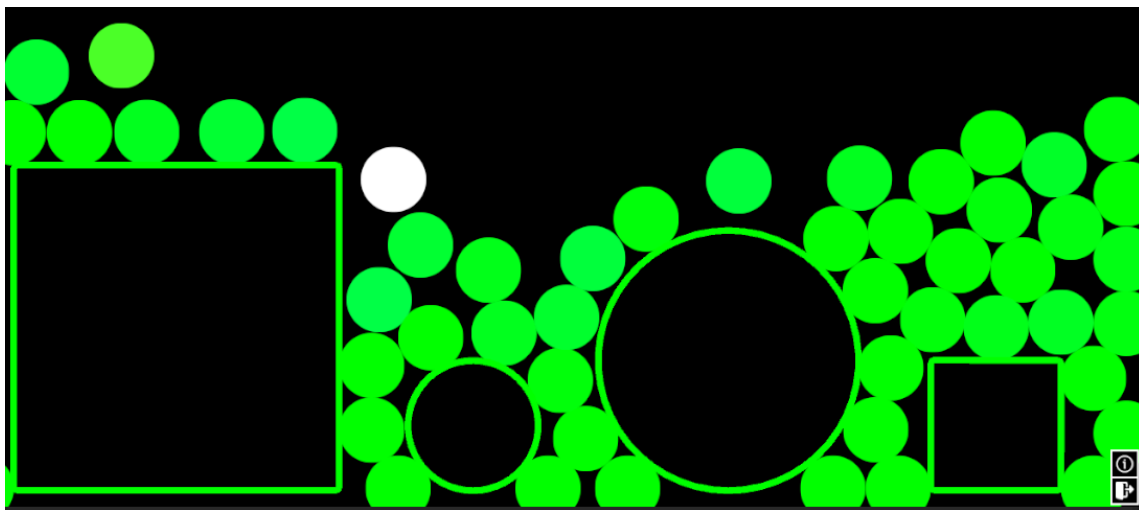


Figure 8: Demonstration of interactions between CPU and GPU Colliders at increased scale for clarity. The outlined colliders run on the CPU, while the solid coloured colliders run on the GPU.

Problems Encountered:

- On slower hardware and repetitive scene reloads, sometimes the memory is not freed completely before the reload causing degraded performance. Suggested to unload the scene entirely, wait for an event indicating that the memory has been freed before reloading the new scene.
 - This issue also appears in Unity 6.
- We're currently trying to switch to Unity 6 from Unity 2022 in order to make our game a web app.
 - Unity 2022 supports cross compiling our game to WebGL however, WebGL does not have the compute shader support we need. Unity 6 supports cross compiling to WebGPU which supports more modern and flexible features such as:
 - Compute Shader support
 - Shared memory between shader stages
 - Atomic operations
 - More flexible memory access patterns
 - Better synchronization primitives
 - Unity 6 also has more robust multi-platform build features, allowing us to effectively target different system implementations.
 - The prevalence of the timing issue with freeing memory is more prevalent in the newer unity version. We need a more robust scene management system before updating.
- Sometimes due to the interparticle forces, many particles can become clumped together in a way that makes them extremely hard to separate, this is an issue because increased density leads to increased neighbourhood checks for particle threads and can cause performance degradation.
- We are currently trying to change our hashing algorithms to a finite hashing scheme. This will allow us to significantly reduce the data transfer cost between the CPU and GPU for fluid simulation by allowing us to only copy the specific particles needed for the CPU to complete the calculations for its particles.

Next Steps:

- Using textures as density maps to make different and more complex collider shapes
- Implementing floating obstacles
- Implementing powdery non-fluid particles like sand
- Hashing algorithm changes
- Levels 3-10
- Bug fixes
- More Optimizations

Conclusion:

At this stage of development, our work on the fluid simulation system has demonstrated promising improvements in CPU-GPU workload distribution and real-time performance. By leveraging Unity's compute shaders and job system, we have enhanced the efficiency of key simulation processes, including particle interactions, temperature diffusion, and state transitions. The implementation of new simulation and gameplay features has provided a solid foundation for future development, though challenges such as interparticle force clumping, inefficient data transfers, and scene memory management persist.

As we move forward, our focus will be on refining data transfer mechanisms, further optimizing compute and memory operations, and expanding the game's interactive features. Future tasks include implementing additional levels, introducing more complex particle behaviors like floating obstacles and granular materials, and transitioning towards Unity 6 and WebGPU for improved cross-platform compatibility. This report serves as a checkpoint in our development process, documenting both our achievements and the technical hurdles that remain, guiding the next phase of iteration and refinement.

References

- [1] R. Budruk, D. Anderson, and T. Shanely, "PCI Express System Architecture," Mindshare.com, [https://www.mindshare.com/files/ebooks/pci express system architecture.pdf](https://www.mindshare.com/files/ebooks/pci%20express%20system%20architecture.pdf) (accessed Feb. 13, 2025).
- [2] Unity Technologies, "Platform-specific rendering," Unity Documentation, 2025. [Online]. Available: <https://docs.unity3d.com/530/Documentation/Manual/SL-PlatformDifferences.html>. [Accessed: Feb. 12, 2025].
- [3] Kodeco, "Unity Job System and Burst Compiler: Getting Started," Kodeco, 2025. [Online]. Available: <https://www.kodeco.com/7880445-unity-job-system-and-burst-compiler-getting-started>. [Accessed: Feb. 12, 2025].
- [4] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation," presented at the 2008 International Conference on Advanced Computing and Analysis Techniques in Physics Research (ACAT), 2008. [Online]. Available: <https://llvm.org/pubs/2008-10-04-ACAT-LLVM-Intro.html>.
- [5] "LLVM Compiler Infrastructure." Accessed: Feb. 12, 2025. [Online]. Available: <https://llvm.org/>

Appendix:

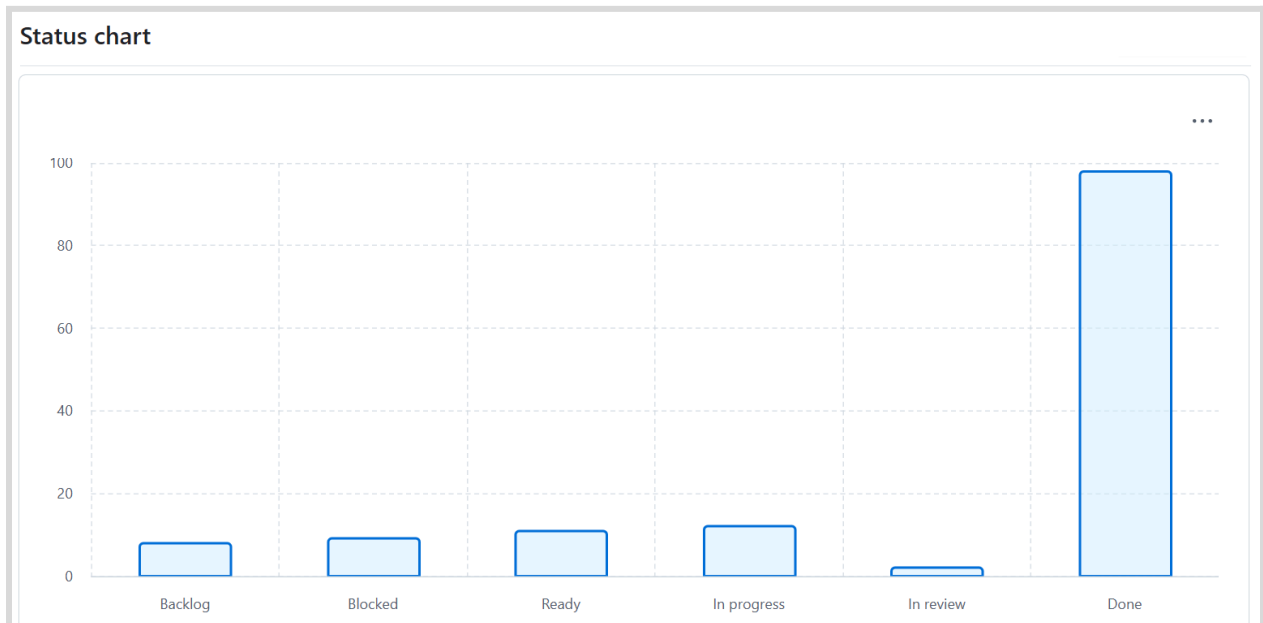


Figure 9: The current status of all tasks on our github repository.
(More Tasks will be created as is necessary)

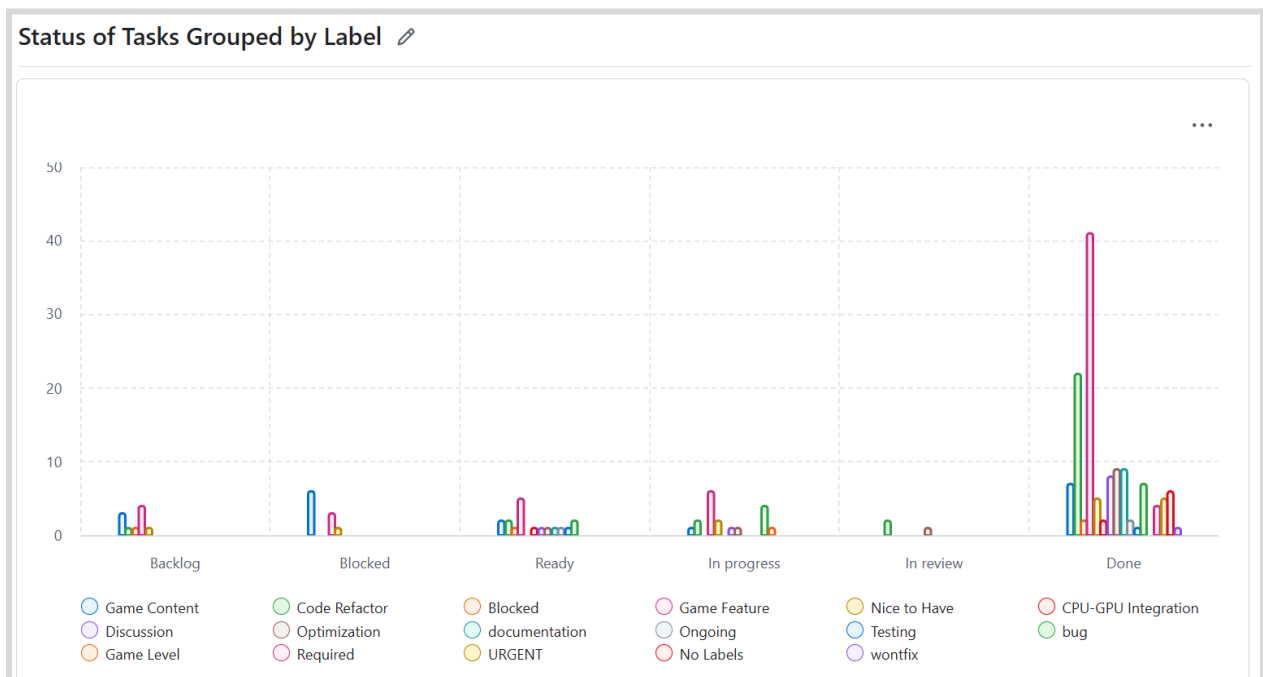


Figure 10: This is the breakdown of the current status of all tasks based on their type. Each bar from left to right corresponds to the legend labels from left to right.

Member Contributions Grouped by Task Type

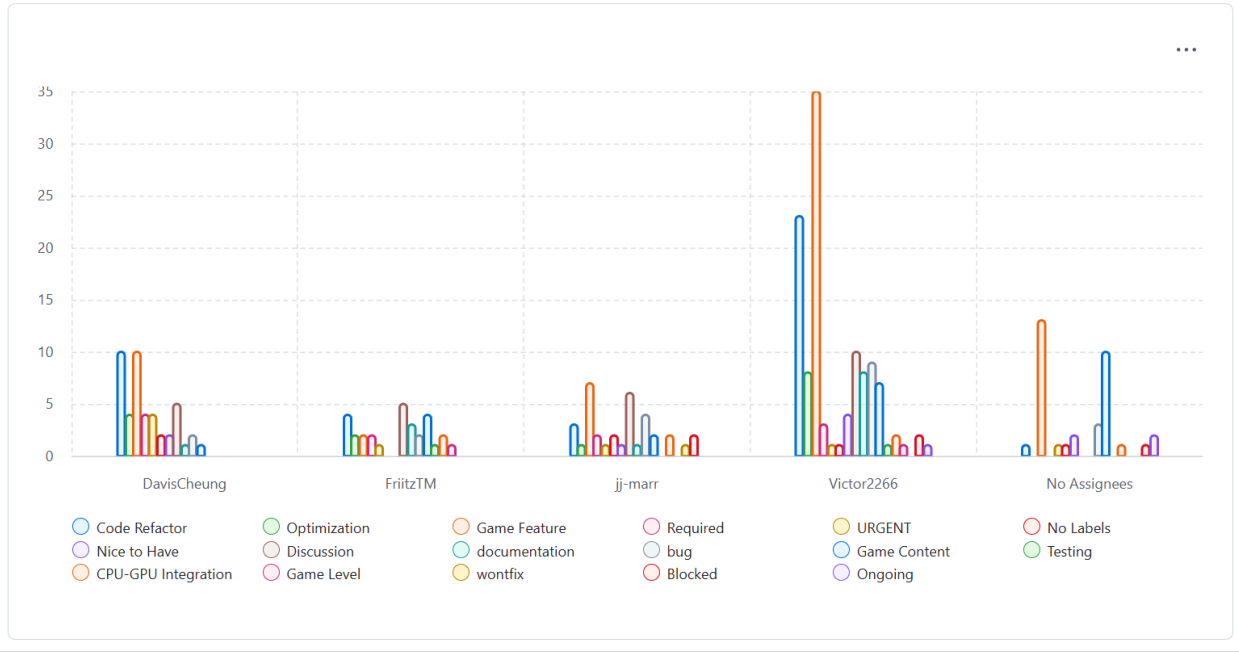


Figure 11: This is a breakdown of the types of tasks each member has worked on grouped by task type.

Member Contributions Grouped by Task Status

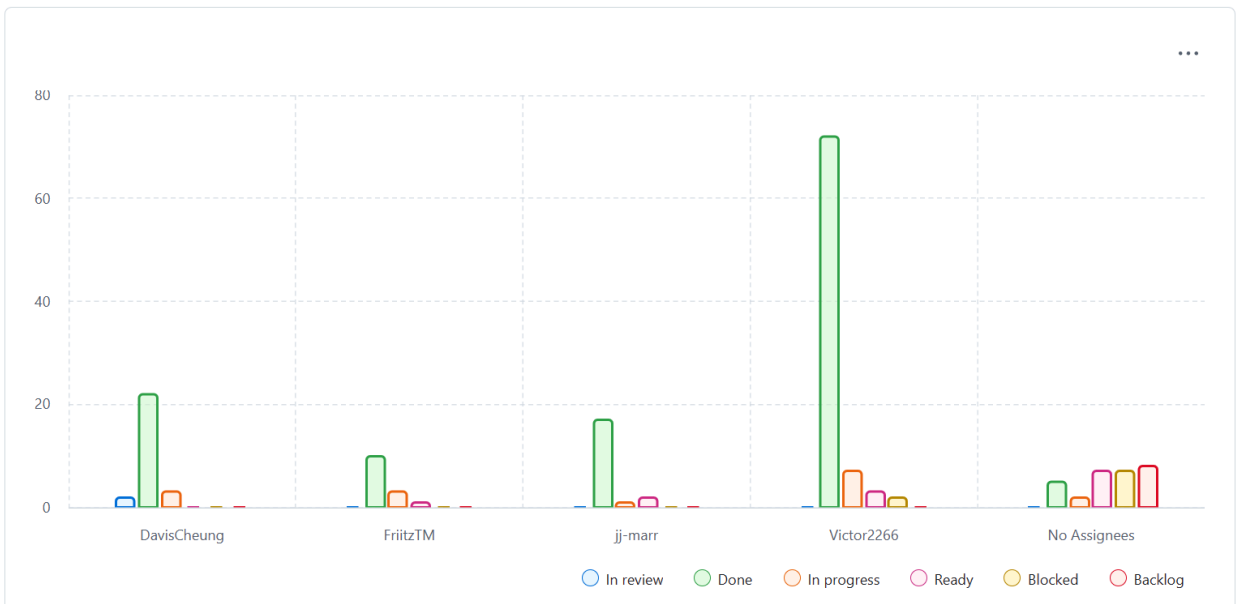


Figure 12: This is a breakdown of the status of tasks assigned to each member.

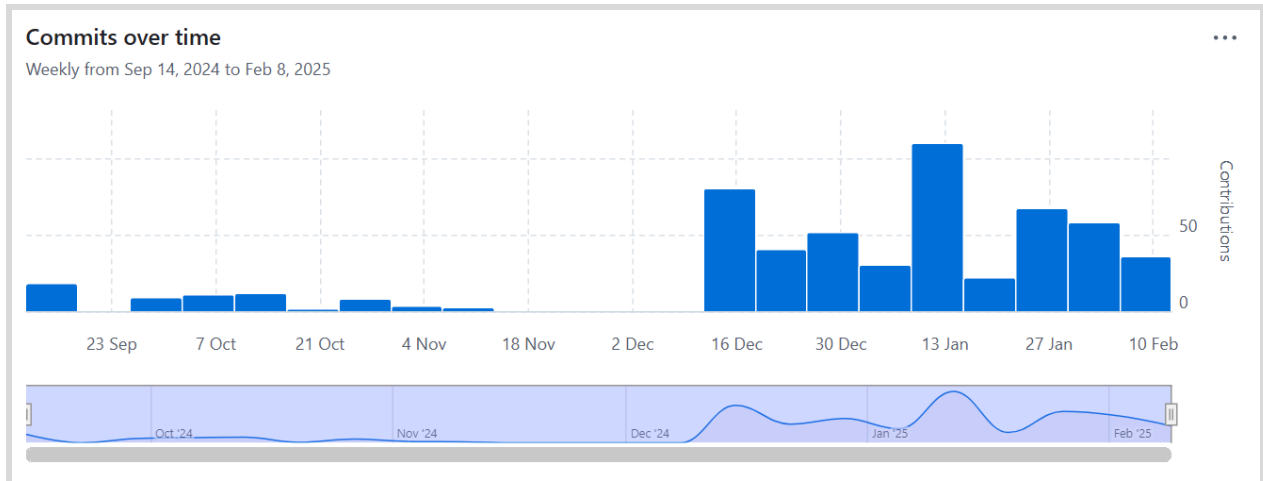


Figure 13: This is the commit history of the project showcasing increased activity after the winter break.

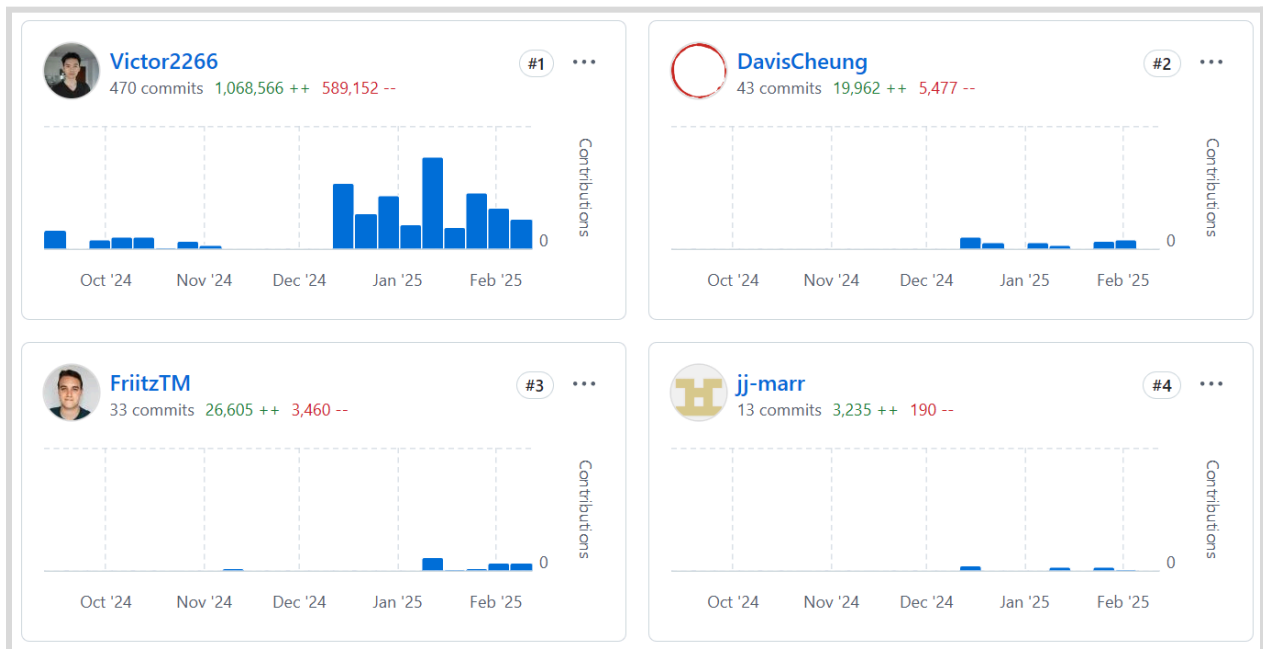


Figure 14: This is a breakdown of the commit history of each member.